

- 2.1** Each process $P_i, i = 1 \dots 9$ is coded as follows repeat

```
P(mutex)
{critical section}
V(mutex)
```

forever

The code for P_{10} is identical except that it uses $V(mutex)$ in place of $P(mutex)$. What is the largest number of processes that can be inside the critical section at any moment?

- (a) 1 (b) 2
(c) 3 (d) None of these

[1997 : 2 M]

- 2.2** When the result of a computation depends on the speed of the processes involved there is said to be
- (a) cycle stealing (b) race condition
(c) a time lock (d) a deadlock

[1998 : 1 M]

- 2.3** A counting semaphore was initialized to 10. Then 6 P (wait) operations and 4V (signal) operations were completed on this semaphore. The resulting value of the semaphore is

- (a) 0 (b) 8
(c) 10 (d) 12

[1998 : 1 M]

- 2.4** Let $m[0] \dots m[4]$ be mutexes (binary semaphores) and $P[0] \dots P[4]$ be processes. Suppose each process $P[i]$ executes the following:

```
wait (m[i]); wait (m[(i+1) mode 4]);
.....
release (m[i]); release (m[(i+1) mod 4]);
```

This could cause

- (a) Thrashing
(b) Deadlock
(c) Starvation, but not deadlock
(d) None of the above

[2000 : 1 M]

- 2.5** Suppose a processor does not have any stack pointer register. Which of the following statement is true?

- (a) It cannot have subroutine call instruction
(b) It can have subroutine call instruction, but no nested subroutine calls
(c) Nested subroutine calls are possible, but interrupts are not
(d) All sequences of subroutine calls and also interrupts are possible

[2001 : 1 M]

- 2.6** Where does the swap space reside?

- (a) RAM (b) Disk
(c) ROM (d) On-chip cache

[2001 : 1 M]

- 2.7** Consider Peterson's algorithm for mutual exclusion between two concurrent processes i and j . The program executed by process i is shown below:

repeat

```
flag[i]=true;
```

```
turn=j;
```

```
while (P) do no-op;
```

```
Enter critical section, perform actions, then
exit critical section
```

```
flag[i]=false;
```

```
Perform other non-critical section actions.
```

Until false;

For the program to guarantee mutual exclusion, the predicate P in the while loop should be

- (a) $\text{flag}[j] = \text{true}$ and $\text{turn} = i$
(b) $\text{flag}[j] = \text{true}$ and $\text{turn} = j$
(c) $\text{flag}[i] = \text{true}$ and $\text{turn} = j$
(d) $\text{flag}[i] = \text{true}$ and $\text{turn} = i$

[2001 : 2 M]

Common Data for Q. 2.8 & 2.9:

Suppose we want to synchronize two concurrent processes P and Q using binary semaphores S and T . The code for the processes P and Q is shown below.

Process P:

```
while (1) {
```

```
W:
```

```
print '0';
```

```
print '0';
```

```
X:
```

```
}
```

Process Q:

```
while (1) {
```

```
Y:
```

```
print '1';
```

```
print '1';
```

```
Z:
```

```
}
```

Synchronization statements can be inserted only at points W, X, Y and Z .

- 2.8** Which of the following will always lead to an output string with '001100110011'?

- (a) $P(S)$ at $W, V(S)$ at $X, P(T)$ at $Y, V(T)$ at Z, S and T initially 1
(b) $P(S)$ at $W, V(T)$ at $X, P(T)$ at $Y, V(S)$ at Z, S initially 1, and T initially 0
(c) $P(S)$ at $W, V(T)$ at $X, P(T)$ at $Y, V(S)$ at Z, S and T initially 1
(d) $P(S)$ at $W, V(T)$ at $X, P(T)$ at $Y, V(T)$ at Z, S initially 1, and T initially 0

[2003 : 2 M]

2.9 Which of the following will ensure that the output string never contains a substring of the form 01^n0 or 10^n1 where n is odd?

- $P(S)$ at W, $V(S)$ at X, $P(T)$ at Y, $V(T)$ at Z, S and T initially 1
- $P(S)$ at W, $V(T)$ at X, $P(T)$ at Y, $V(S)$ at Z, S and T initially 1
- $P(S)$ at W, $V(S)$ at X, $P(S)$ at Y, $V(S)$ at Z, S initially 1
- $V(S)$ at W, $V(T)$ at X, $P(S)$ at Y, $P(T)$ at Z, S and T initially 1

[2003 : 2 M]

2.10 Consider two processes P_1 and P_2 accessing the shared variables X and Y protected by two binary semaphores S_X and S_Y respectively, both initialized to 1. P and V denote the usual semaphore operators, where P decrements the semaphore value, and V increments the semaphore value. The pseudocode of P_1 and P_2 is as follows:

P_1 : while true do { L_1 : L_2 : $X = X + 1$; $Y = Y - 1$; $V(S_X)$; $V(S_Y)$; }	P_2 : while true do { L_3 : L_4 : $Y = Y + 1$; $X = X - 1$; $V(S_Y)$; $V(S_X)$; }
--	--

In order to avoid deadlock, the correct operators at L_1, L_2, L_3 and L_4 are respectively

- $P(S_Y), P(S_X), P(S_X), P(S_Y)$
- $P(S_X), P(S_Y), P(S_Y), P(S_X)$
- $P(S_X), P(S_X), P(S_Y), P(S_Y)$
- $P(S_X), P(S_Y), P(S_X), P(S_Y)$

[2004 : 2 M]

2.11 The semaphore variables full, empty and mutex are initialized to 0, n and 1, respectively. Process P_1 repeatedly adds one item at a time to a buffer of size n, and process P_2 repeatedly removes one item at a time from the same buffer using the programs given below. In the programs, K, L, M and N are unspecified statements.

P_1 : while (1)

```
{
    K;
    P(mutex);
    Add an item to the buffer;
    V(mutex);
    L;
}
```

P_2 : while (1)

```
{
    M;
    P(mutex);
    Remove an item from the buffer;
    V(mutex);
    N;
}
```

The statements K, L, M and N are respectively

- $P(\text{full}), V(\text{empty}), P(\text{full}), V(\text{empty})$
- $P(\text{full}), V(\text{empty}), P(\text{empty}), V(\text{full})$
- $P(\text{empty}), V(\text{full}), P(\text{empty}), V(\text{full})$
- $P(\text{empty}), V(\text{full}), P(\text{full}), V(\text{empty})$

[2004 : 2 M]

2.12 Given below is a program which when executed spawns two concurrent processes:

semaphore X := 0 ;

/* Process now forks into concurrent processes P_1 & P_2 */

P_1 : repeat forever $V(X)$; Compute; $P(X)$;	P_2 : repeat forever $P(X)$; Compute; $V(X)$;
--	--

Consider the following statements about processes P_1 and P_2 :

- It is possible for process P_1 to starve.
- It is possible for process P_2 to starve.

Which of the following holds?

- Both I and II are true.
- I is true but II is false.
- II is true but I is false
- Both I and II are false.

[2005 : 2 M]

2.13 Two concurrent processes P_1 and P_2 use four shared resources R1, R2, R3 and R4, as shown below:

P_1 : Compute: Use R1; Use R2; Use R3; Use R4;	P_2 : Compute; Use R1; Use R2; Use R3; Use R4;
---	---

Both processes are started at the same time, and each resource can be accessed by only one process at a time. The following scheduling constraints exist between the access of resources by the processes:

- P_2 must complete use of R1 before P_1 gets access to R1.
- P_1 must complete use of R2 before P_2 gets access to R2.

- P2 must complete use of R3 before P1 gets access to R3.
- P1 must complete use of R4 before P2 gets access to R4.

There are no other scheduling constraints between the processes. If only binary semaphores are used to enforce the above scheduling constraints, what is the minimum number of binary semaphores needed?

- (a) 1 (b) 2
(c) 3 (d) 4 [2005 : 2 M]

2.14 The atomic *fetch-and-set* x, y instruction unconditionally sets the memory location x to 1 and fetches the old value of x in y without allowing any intervening access to the memory location x . Consider the following implementation of P and V functions on a binary semaphore S.

```
void P(binary_semaphore *S) {
    unsigned y;
    unsigned *x = &(S → value);
    do {
        fetch-and-set x, y;
    } while (y);
}

void V(binary_semaphore *S) {
    S → value = 0;
}
```

Which one of the following is true?

- (a) The implementation may not work if context switching is disabled in P
(b) Instead of using *fetch-and-set*, a pair of normal load/store can be used
(c) The implementation of V is wrong
(d) The code does not implement a binary semaphore

[2006 : 2 M]

Common Data for Q. 2.15 & 2.16:

Barrier is a synchronization construct where a set of processes synchronizes globally i.e. each process in the set arrives at the barrier and waits for all others to arrive and then all processes leave the barrier. Let the number of processes in the set be three and S be a binary semaphore with the usual P and V functions. Consider the following C implementation of a barrier with line numbers shown on the left.

```
void barrier(void) {
1 : P(S);
2 : process_arrived++;
3 : V(S);
4 : while (process_arrived != 3);
5 : P(S);
```

```
6 : process_left++;
7 : if (process_left == 3){
8 :     process_arrived = 0;
9 :     process_left = 0;
10 : }
11 : V(S);
12 : }
```

The variables *process_arrived* and *process_left* are shared among all processes and are initialized to zero. In a concurrent program all the three processes call the barrier function when they need to synchronize globally.

2.15 The above implementation of barrier is incorrect. Which one of the following is true?

- (a) The barrier implementation is wrong due to the use of binary semaphore S
(b) The barrier implementation may lead to a deadlock if two barrier invocations are used in immediate succession
(c) Lines 6 to 10 need not be inside a critical section
(d) The barrier implementation is correct if there are only two processes instead of three

[2006 : 2 M]

2.16 Which one of the following rectifies the problem in the implementation?

- (a) Lines 6 to 10 are simply replaced by *process_arrived*
(b) At the beginning of the barrier the first process to enter the barrier waits until *process_arrived* becomes zero before proceeding to execute P(S)
(c) Context switch is disabled at the beginning of the barrier and re-enabled at the end
(d) The variable *process_left* is made private instead of shared

[2006 : 2 M]

2.17 Processes P1 and P2 use *critical_flag* in the following routine to achieve mutual exclusion. Assume that *critical_flag* is initialized to FALSE in the main program.

```
get_exclusive_access()
{
    if (critical_flag == FALSE)
    {
        critical_flag = TRUE;
        critical_region();
        critical_flag = FALSE;
    }
}
```

Consider the following statements.

(i) It is possible for both P1 and P2 to access critical_region concurrently.

(ii) This may lead to a deadlock.

Which of the following holds?

(a) (i) is false and (ii) is true

(b) Both (i) and (ii) are false

(c) (i) is true and (ii) is false

(d) Both (i) and (ii) are true

[2007 : 1 M]

2.18 Two processes, P1 and P2, need to access a critical section of code. Consider the following synchronization construct used by the processes:

```
/* P1 */
while (true) {
    wants1 = true;
    while (wants2 == true);
    /* Critical
       Section */
    wants1 = false;
}
/* Remainder section */
```

```
/* P2 */
while (true) {
    wants2 = true;
    while (wants1 == true);
    /* Critical
       Section */
    wants2 = false;
}
/* Remainder section */
```

Here, wants1 and wants2 are shared variables, Which are initialized to false. Which one of the following statements is TRUE about the above construct?

(a) It does not ensure mutual exclusion.

(b) It does not ensure bounded waiting.

(c) It requires that processes enter the critical section in strict alternation.

(d) It does not prevent deadlocks, but ensures mutual exclusion.

[2007 : 2 M]

2.19 Synchronization in the classical readers and writers problem can be achieved through use of semaphores. In the following incomplete code for readers-writers problem, two binary semaphores mutex and wrt are used to obtain synchronization

wait (wrt)

writing is performed

signal (wrt)

wait (mutex)

readcount = readcount + 1

if readcount = 1 then S1

S2

reading is performed

S3

readcount = readcount - 1

if readcount = 0 then S4

Signal (mutex)

The values of S1, S2, S3, S4, (in that order) are

(a) signal (mutex), wait (wrt), signal (wrt), wait (mutex)

(b) signal (wrt), signal (mutex), wait (mutex), wait (wrt)

(c) wait (wrt), signal (mutex), wait (mutex), signal (wrt)

(d) signal (mutex), wait (mutex), signal (mutex), wait (mutex)

[2007 : 2 M]

2.20 The P and V operations on counting semaphores, where s is a counting semaphore, are defined as follows:

P(s): s = s - 1;

if s < 0 then wait;

V(s): s = s + 1;

if s ≤ 0 then wakeup a process waiting on s;

Assume that P_b and V_b, the wait and signal operations on binary semaphores are provided.

Two binary semaphores X_b and Y_b are used to implement the semaphore operations P(s) and V(s) as follows:

P(s): P_b(X_b);

s = s - 1;

if (s < 0) {

V_b(X_b);

P_b(Y_b);

}

else V_b(X_b);

V(s): P_b(X_b);

s = s + 1;

if (s ≤ 0) V_b(Y_b);

V_b(X_b);

The initial values of X_b and Y_b are respectively

(a) 0 and 0

(b) 0 and 1

(c) 1 and 0

(d) 1 and 1 [2008 : 2 M]

2.21 The enter_CS () and leave_CS () functions to implement critical section of a process are realized using test-and-set instruction as follows:

Void enter_CS (X)

{

while (test-and-set (X));

}

Void leave_CS (X)

{

X = 0 ;

}

In the above solution, X is a memory location associated with the CS and is initialized to 0. Now consider the following statements

I. The above solution to CS problem is deadlock-free.

- II. The solution is starvation free.
 III. The processes enter CS in FIFO order.
 IV. More than one process can enter CS at the same time.

Which of the above statements are TRUE?

- (a) I only (b) I and II
 (c) II and III (d) IV only [2009 : 2 M]

2.22 Consider the methods used by processes P1 and P2 for accessing their critical sections whenever needed, as given below. The initial values of shared boolean variables S1 and S2 are randomly assigned.

Method used by P1	Method used by P2
while (S1==S2); Critical Section S1=S2;	while (S1!=S2); Critical Section S2=not (S1);

Which one of the following statements describes the properties achieved?

- (a) Mutual exclusion but not progress
 (b) Progress but not mutual exclusion
 (c) Neither mutual exclusion nor progress
 (d) Both mutual exclusion and progress

[2010 : 1 M]

2.23 The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as S0 = 1, S1 = 0, S2 = 0.

Process P0	Process P1	Process P2
while (true) { wait (S0); print '0' release (S1); release (S2); }	wait (S1); release(S0);	wait (S2) release(S0);

How many times will process P0 print '0'?

- (a) At least twice (b) Exactly twice
 (c) Exactly thrice (d) Exactly once

[2010 : 2 M]

2.24 Fetch_And_Add(X, i) is an atomic Read-Modify-Write instruction that reads the value of memory location X, increments it by the value i, and returns the old value of X, it is used in the pseudocode shown below to implement a busy-wait lock. L is unsigned integer shared variable initialized to 0. The value of 0 corresponds to lock being available, while any non-zero value corresponds to the lock being not available.

```
AcquireLock (L){
    while ( Fetch_And_Add(L, 1) )
```

```
    L = 1;
}
ReleaseLock (L){
    L = 0;
}
```

This implementation

- (a) fails as L can overflow
 (b) fails as L can take on a non-zero value when the lock is actually available
 (c) works correctly but may starve some processes
 (d) works correctly without starvation

[2012 : 2 M]

2.25 Three concurrent processes X, Y and Z execute three different code segments that access and update certain shared variables. Process X execute the P operation (i.e., wait) on semaphores a, b and c; process Y executes the P operation on semaphores b, c and d; process Z executes the P operation on semaphores c, d and a before entering the respective code segments. After completing the execution of its code segments, each process invokes the V operation (i.e., signal) on its three semaphores. All semaphores are binary semaphores initialized to one. Which one of the following represents a deadlock-free order of invoking the P operations by the processes?

- (a) X : P(a)P(b)P(c) (b) X : P(b)P(a)P(c)
 Y : P(b)P(c)P(d) Y : P(b)P(c)P(d)
 Z : P(c)P(d)P(a) Z : P(a)P(c)P(d)
 (c) X : P(b)P(a)P(c) (d) X : P(a)P(b)P(c)
 Y : P(c)P(b)P(d) Y : P(b)P(c)P(d)
 Z : P(a)P(c)P(d) Z : P(c)P(d)P(a)

[2013 : 1 M]

2.26 A shared variable x, initialized to zero, is operated on by four concurrent process W, X, Y, Z as follows. Each of the processes W and X reads x from memory, increments by one, stores it to memory, and then terminates. Each of the processes Y and Z reads x from memory, decrements by two, stores it to memory and then terminates. Each process before reading x invokes the P operations (i.e., wait) on a counting semaphore S and invokes the V operation (i.e., signal) on the semaphore S after storing x to memory. Semaphore S is initialized to two. What is the maximum possible value of x after all processes complete execution?

- (a) -2 (b) -1
 (c) 1 (d) 2

[2013 : 2 M]

2.27 A certain computation generates two arrays a and b such that $a[i] = f$ for $0 \leq i \leq n$ and $b[i] = g(a[i])$ for $0 \leq i < n$. Suppose this computation is decomposed into two concurrent processes X and Y such that X computes the array a and Y computes the array b . The processes employ two binary semaphores R and S , both initialized to zero. The array a is shared by the two processes. The structure of the processes are shown below:

Process X	Process Y
private i;	private i;
for (i = 0; i < n; i++)	for (i = 0; i < n; i++)
{	{
$a[i] = f(i);$	EntryY(R, S);
ExitX(R, S);	$b[i] = g(a[i]);$
}	}

Which one of the following represents the **CORRECT** implementations of ExitX and EntryY?

- | | |
|---|---|
| (a) ExitX(R, S) {
P(R);
V(S);
}
EntryY(R, S) {
P(S);
V(R);
} | (b) ExitX(R, S) {
V(R);
V(S);
}
EntryY(R, S) {
P(R);
P(S);
} |
| (c) ExitX(R, S) {
P(S);
V(R);
}
EntryY(R, S) {
V(S);
P(R);
} | (d) ExitX(R, S) {
V(R);
P(S);
}
EntryY(R, S) {
V(S);
P(R);
} |

[2013 : 2 M]

2.28 Consider the procedure below for the Producer-Consumer problem which uses semaphores:

```

semaphore n = 0;
semaphore s = 1;

void producer()
{
    while(true)
    {
        produce();
        semWait(s);
        addToBuffer();
        semSignal(s);
        semSignal(n);
    }
}

void consumer()
{
    while (true)
    {
        semWait(s);
        semWait(n);
        removeFromBuffer();
        semSignal(s);
        consume();
    }
}

```

Which one of the following is TRUE?

- The producer will be able to add an item to the buffer, but the consumer can never consume it.
- The consumer will remove no more than one item from the buffer.
- Deadlock occurs if the consumer succeeds in acquiring semaphore s when the buffer is empty.
- The starting value for the semaphore n must be 1 and not 0 for deadlock-free operation.

[2014 (Set-2) : 2 M]

2.29 The following two functions P1 and P2 that share a variable B with an initial value of 2 execute concurrently.

P1()	P2()
{	{
$C = B - 1;$	$D = 2 * B;$
$B = 2 * C;$	$B = D - 1;$
}	}

The number of distinct values that B can possibly take after the execution is _____.

[2015 (Set-1) : 1 M]

2.30 Two processes X and Y need to access a critical section. Consider the following synchronization construct used by both the processes

Process X
/* other code for process X */
while (true)
{
 varP = true;
 while (varQ == true)
 { /* Critical Section */
 varP = false;
 }
}
/* other code for process X */

Process Y
/* other code for process Y */
while (true)
{
 varQ = true;
 while (varP == true)
 { /* Critical Section */
 varQ = false;
 }
}
/* other code for process Y */

Here, varP and varQ are shared variables and both are initialized to false. Which one of the following statements is true?

- The proposed solution prevents deadlock but fails to guarantee mutual exclusion
- The proposed solution guarantees mutual exclusion but fails to prevent deadlock

- (c) The proposed solution guarantees mutual exclusion and prevents deadlock
 (d) The proposed solution fails to prevent deadlock and fails to guarantee mutual exclusion

[2015 (Set-3) : 1 M]

2.31 Consider the following proposed solution for the critical section problem. There are n processes: $P_0 \dots P_{n-1}$. In the code, function $pmax$ returns an integer not smaller than any of its arguments. For all i , $t[i]$ is initialized to zero.

Code for P_i :

```
do
{
   $c[i]=1$ ;  $t[i] = pmax(t[0], \dots, t[n-1]) + 1$ ;  $c[i] = 0$ ;
  for every  $j \neq i$  in  $\{0, \dots, n-1\}$ 
  { while ( $c[j]$ );
    while ( $t[j] \neq 0 \ \&\& \ t[j] \leq t[i]$ );
  }
  Critical Section;  $t[i] = 0$ ;
  Remainder Section;
} while (true);
```

Which one of the following is TRUE about the above solution?

- (a) At most one process can be in the critical section at any time
 (b) The bounded wait condition is satisfied
 (c) The progress condition is satisfied
 (d) It cannot cause a deadlock

[2016 (Set-1) : 2 M]

2.32 Consider the following two-process synchronization solution:

Process 0

Entry: loop while ($turn == 1$);
 (critical section)
 Exit: $turn = 1$;

Process 1

Entry : loop while ($turn == 0$);
 (critical section)
 Exit: $turn = 0$;

The shared variable $turn$ is initialized to zero. Which one of the following is TRUE?

- (a) This is a correct two-process synchronization solution
 (b) This solution violates mutual exclusion requirement
 (c) This solution violates progress requirement
 (d) This solution violates bounded wait requirement

[2016 (Set-2) : 2 M]

2.33 Consider a non-negative counting semaphore S . The operation $P(S)$ decrements S , and $V(S)$ increments S . During an execution, 20 $P(S)$ operations and 12 $V(S)$ operations are issued in some order. The largest initial value of S for which at least one $P(S)$ operation will remain blocked is _____. [2016 (Set-2) : 2 M]

2.34 A multithreaded program P executes with x number of threads and uses y number of locks for ensuring mutual exclusion while operating on shared memory locations. All locks in the program are non-reentrant, i.e., if a thread holds a lock l . Then it cannot re-acquire lock l without releasing it. If a thread is unable to acquire a lock, it blocks until the lock becomes available. The minimum value of x and the minimum value of y together for which execution of P can result in a deadlock are:

- (a) $x = 1, y = 2$
 (b) $x = 2, y = 1$
 (c) $x = 2, y = 2$
 (d) $x = 1, y = 1$

[2017 (Set-1) : 2 M]

2.35 Consider the following solution to the producer-consumer synchronization problem. The shared buffer size is N . Three semaphores $empty$, $full$ and $mutex$ are defined with respective initial values of 0, N and 1. Semaphore $empty$ denotes the number of available slots in the buffer, for the consumer to read from. Semaphore $full$ denotes the number of available slots in the buffer, for the producer to write to. The placeholder variables, denoted by P , Q , R and S , in the code below can be assigned either $empty$ or $full$. The valid semaphore operations are: wait () and signal ().

Producer:	Consumer
do { wait (P); wait (mutex); //Add item to buffer signal (mutex); signal (Q); } while (1);	do { wait (R); wait (mutex); //Consume item from buffer signal (mutex); signal (S); } while (1);

Which one of the following assignment to P , Q , R and S will yield the correct solution?

- (a) P : full, Q : full, R : empty, S : empty
 (b) P : empty, Q : empty, R : full, S : full
 (c) P : full, Q : empty, R : empty, S : full
 (d) P : empty, Q : full, R : full, S : empty

[2018 : 2 M]

- 2.36 Consider three concurrent processes P1, P2 and P3 as shown below, which access a shared variable D that has been initialized to 100.

P1	P2	P3
:	:	:
:	:	:
$D = D + 20$	$D = D - 50$	$D = D + 10$
:	:	:
:	:	:

The processes are executed on a uniprocessor system running a time-shared operating system. If the minimum and maximum possible values of D after the three processes have completed executed are X and Y respectively, then the value of $Y - X$ is _____. [2019 : 1 M]

- 2.37 Consider the following snapshot of a system running n concurrent processes. Process i is holding X_i instances of a resource R, $1 \leq i \leq n$. Assume that all instances of R are currently in use. Further, for all i , process i can place a request for at most Y_i additional instances of R while holding the X_i instances it already has. Of the n processes, there are exactly two processes p and q such that $Y_p = Y_q = 0$. Which one of the following conditions guarantees that no other process apart from p and q can complete execution?

- $X_p + X_q < \text{Min} \{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
- $\text{Min} (X_p, X_q) \geq \text{Min} \{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
- $\text{Min} (X_p, X_q) \leq \text{Max} \{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$
- $X_p + X_q < \text{Max} \{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$

[2019 : 2 M]

- 2.38 Each of a set of n processes executes the following code using two semaphores a and b initialized to 1 and 0, respectively. Assume that count is a shared variable initialized to 0 and not used in CODE SECTION P.

CODE SECTION P

```
wait(a); count = count + 1;
if (count == n) signal(b);
signal(a); wait(b); signal(b);
```

CODE SECTION Q

What does the code achieve?

- It ensures that at most two processes are in CODE SECTION Q at any time.
- It ensures that all processes execute CODE SECTION P mutually exclusively.
- It ensures that no process executes CODE SECTION Q before every process has finished CODE SECTION P.

- It ensures that at most $n - 1$ processes are in CODE SECTION P at any time.

[2020 : 2 M]

- 2.39 Consider the following pseudocode, where S is a semaphore initialized to 5 in line#2 and counter is a shared variable initialized to 0 in line#1. Assume that the increment operation in line#7 is not atomic.

```
1. int counter = 0;
2. Semaphore S = init(5);
3. void parop(void)
4. {
5.     wait(S);
6.     wait(S);
7.     counter++;
8.     signal(S);
9.     signal(S);
10. }
```

If five threads execute the function parop concurrently, which of the following program behavior(s) is/are possible?

- The value of counter is 1 after all the threads successfully complete the execution of parop.
- The value of counter is 0 after all the threads successfully complete the execution of parop.
- There is a deadlock involving all the threads.
- The value of counter is 5 after all the threads successfully complete the execution of parop.

[2021 (Set-1) : 2 M]

- 2.40 Consider a computer system with multiple shared resource types, with one instance per resource type. Each instance can be owned by only one process at a time. Owning and freeing of resources are done by holding a global lock (L). The following scheme is used to own a resource instance:

function OwnResource (Resource R)

Acquire lock L // a global lock

if R is available then

Acquire R

Release lock L

else

if R is owned by another process P then

Terminate P, after releasing all

resources owned by P

Acquire R

Restart P

Release lock L

end if

end if

end function

Which of the following choice(s) about the above scheme is/are correct?

- (a) The scheme violates the mutual exclusion property.
- (b) The scheme ensures that deadlocks will not occur.
- (c) The scheme may lead to live-lock.
- (d) The scheme may lead to starvation.

[2021 (Set-2) : 2 M]

2.41 Consider the following multi-threaded code segment (in a mix of C and pseudo-code), invoked by two processes P_1 and P_2 , and each of the processes spawns two threads T_1 and T_2 :

```
int x = 0;      // global
Lock L1;       // global
main() {
    create a thread to execute foo(); // Thread  $T_1$ 
    create a thread to execute foo(); // Thread  $T_2$ 
    wait for the two threads to finish execution;
    print (x);}
foo() {
    int y = 0;
    Acquire  $L_1$ ;
    x = x + 1;
    y = y + 1;
    Release  $L_1$ ;
    print (y);}
```

Which of the following statement(s) is/are correct?

- (a) At least one of P_1 and P_2 will print the value of x as 4,
- (b) Both P_1 and P_2 will print the value of x as 2.
- (c) Both T_1 and T_2 , in both the processes, will print the value of y as 1.
- (d) At least one of the threads will print the value of y as 2.

[2021 (Set-2) : 2 M]

2.42 Consider the following threads, T_1 , T_2 and T_3 executing on a single processor, synchronized using three binary semaphore variables, S_1 , S_2 and S_3 , operated upon using standard wait() and signal(). The threads can be context switched in any order and at any time.

T_1	T_2	T_3
while (true) { wait (S_3); print ("C"); signal (S_2); }	while (true) { wait (S_1); print ("B"); signal (S_3); }	while (true) { wait (S_2); print ("A"); signal (S_1); }

Which initialization of the semaphores would print the sequence BCABCABCA

- (a) $S_1 = 1; S_2 = 1; S_3 = 1$
- (b) $S_1 = 1; S_2 = 1; S_3 = 0$
- (c) $S_1 = 1; S_2 = 0; S_3 = 0$
- (d) $S_1 = 0; S_2 = 1; S_3 = 1$

[2022 : 1 M]

2.43 Consider the two functions incr and decr shown below:

```
incr() {                                decr() {
    wait(s);                            wait(s);
    X = X + 1;                          X = X - 1;
    signal(s);                          signal(s);
}
```

There are 5 threads each invoking incr once, and 3 threads each invoking decr once, on the same shared variable X . The initial value of X is 10. Suppose there are two implementations of the semaphore s , as follows:

I-1: s is a binary semaphore initialized to 1.

I-2: s is a counting semaphore initialized to 2.

Let V_1 , V_2 be the values of X at the end of execution of all the threads with implementations I-1, I-2, respectively.

Which one of the following choices corresponds to the minimum possible values of V_1 , V_2 , respectively?

- (a) 15, 7
- (b) 7, 7
- (c) 12, 7
- (d) 12, 8

[2023 : 2 M]

2.44 Consider the following code snippet using the fork() and wait() system calls. Assume that the code compiles and runs correctly, and that the system calls run successfully without any errors.

```
int x = 3;
while (x > 0) {
    fork( );
    printf("hello");
    wait(NULL);
    x--;
}
```

The total number of times the printf statement is executed is _____.

[2024 (Set-1) : 2 M]

2.45 Consider three processes P1, P2, and P3 running identical code, as shown in the pseudocode below. A and B are two binary semaphores initialized to 1 and 0, respectively. X is a shared variable initialized to 0. Each line in the pseudocode is executed atomically.

Pseudocode of P1, P2, and P3

```
Wait(A);
```

```
Print(*);
```

```
X = X + 1;
```

```
If (X == 2)
```

```
{
```

```
    Print($);
```

```
    Signal(B);
```

```
}
```

```
Signal(A);
```

```
Wait(B);
```

```
Print(#);
```

```
Signal(B);
```

Assume that any of the three processes can start to execute first and context switching can happen between these processes at any arbitrary time and in any arbitrary order.

Which of the following patterns is/are possible to be generated as an outcome of the execution of these three processes?

(a) **\$*###

(b) **\$*###

(c) **\$*###*

(d) ***\$###

[2026 (Set-2) : 2 M]

■■■■

Answers Process Management-II

2.1 (d)	2.2 (b)	2.3 (b)	2.4 (b)	2.5 (a)	2.6 (b)	2.7 (b)	2.8 (b)	2.9 (c)
2.10 (d)	2.11 (d)	2.12 (d)	2.13 (b)	2.14 (a)	2.15 (b)	2.16 (b)	2.17 (c)	2.18 (d)
2.19 (c)	2.20 (c)	2.21 (a)	2.22 (a)	2.23 (a)	2.24 (b)	2.25 (b)	2.26 (d)	2.27 (c)
2.28 (c)	2.29 (3)	2.30 (a)	2.31 (a)	2.32 (c)	2.33 (7)	2.34 (d)	2.35 (c)	2.36 (80)
2.37 (a)	2.38 (c)	2.39 (a, c, d)		2.40 (b, c, d)		2.41 (b, c)	2.42 (c)	2.43 (c)
2.44 (14)	2.45 (a, b, c)							

Explanations Process Management-II**2.1 (d)**

Everytime when process P_{10} enters it performs the 'UP' operation so after that one more process can enter into it, so this phenomena continue till end. So 10 processes at a time can enter into critical section.

2.2 (b)

Race condition.

2.3 (b)

Initially semaphore is 10, then 6 down operations are performed means $(10 - 6)$ and 4 up operation means $(4 + 4) = 8$.

So option (b) is correct.

2.4 (b)

All processes are in deadlock condition i.e., not a single one process can enter into critical section.

2.5 (a)

If stack pointer register is not available then activation records in the stack can not be created. So it cannot have subroutine call instruction.

2.6 (b)

Disk has swap space.

2.7 (b)

To ensure the mutual exclusion, we have to take care that 'turn' value which is 'j' so we should not allow that process in CS which is having flag $[j] = \text{true}$.

Hence option (b) is correct.

2.8 (b)

To get the expected output, The sequence of processes are P, Q, P, Q, P, Q and so on we must initialize $S = 1$ and $T = 0$.

2.9 (c)

If the output string never contains a substring of the form 01^n0 or 10^n1 where n is odd then the procedures becomes.

$S = 1$

Process P: Process Q:

While(1){ While(1){

W : P(S) Y : P(S)

print '0'; print '1';

print '0'; print '1';

X : V(S) Z : V(S)

}

}

2.10 (d)

S_x and S_y are two binary semaphores if we assume P means wait and V means signal then for two process P_1 and P_2 we take alternate then there in no chance of dead lock.

2.11 (d)

Here full and empty are integer semaphore which denote number of full slots in buffer and number of empty slot in buffer at any time respectively. While mutex is binary semaphore to provide mutual exclusion.

Initially full = 0 empty = n and mutex = 1.

- When no item into buffer, since mutex = 1. So to restrict consumer to remove item P (full) must be there at M and when it leave the critical section made V (empty) at N i.e. one empty slot increase.
- When producer tries to add an item into buffer before entering to critical section, it must perform P (empty) i.e. one empty decrease and while leaving critical section make V (full) i.e. one new item added successfully.

So, correct order will be P (empty), V (full), P (full) and V (empty) to ensure consumer will be block until 1 item added into buffer.

2.12 (d)

None of the processes will ever starve. Starvation is said to be possible if in ALL the possible ways that a process want to enter CS, it is DENIED the access. But that can't happens here. P1 is never starved because this is INFACHT the first process that can enter. V(X) by P1 is always successful and hence it always enter the CS. So 1 is FALSE.
When P1 is in CS, X = 1. At this time P2 can make a successful DOWN operation P(X) and enter CS. So 2 is also FALSE.
But P2 must exit the CS before P1.

2.13 (b)

With one semaphore it is not possible, since binary semaphore is used.
So, try with 2 binary semaphore a and b assume initialize to 0.

P1:	P2:
P(a) R1	R1 V(a) P(b) R2
R2 V(b) P(a) R3	R3 V(a) P(b) R4
R4 V(b)	

So, minimum 2 semaphore needed.

2.14 (a)

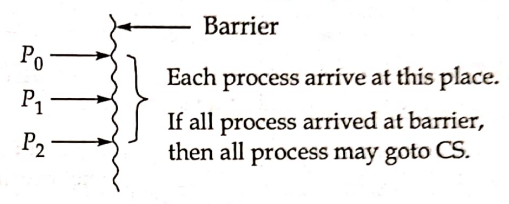
The P and V function is as follows:

```

1. void P (binary-semaphore * S){
2.     unsigned y;
3.     unsigned * x = & (S → value);
4.     do {
5.         fetch-and-set x, y;
6.     } while (y);
7. }
8. void V (binary-semaphore * S){
9.     S → value = 0;
10. }
    
```

fetch-and-set instruction always sets the memory location $x = 1$ and fetches the old value of x into y . The binary semaphore * S takes only two values either 0 and 1. When we initiliaze $S = 0$ then in statement 3 this value will be start at location x and fetch-and-set instruction changes the value of $x = 0$ to $x = 1$ and y becomes 0. If there are more than two processes and context switching between processes is disabled in P then this implementation doesn't work properly and can't synchronize the processes.

2.15 (b)



Given code:

```

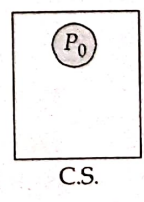
1. P(S);
2. process_arrived ++;
3. V(S);
4. while (process_arrived != 3);
5. P(S);
6. process_left ++;
7. if (process_left == 3)
8. { process_arrived = 0;
9.   process_left = 0;
10. }
11. V(S);
    
```

This time act as barrier

These code work like critical section

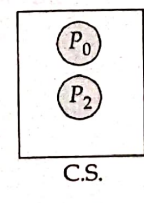
Each process in the set arrive at barrier. We need to provide synchronization to process to enter barrier and wait for all other processes to arrive. Once they arrive, they start leaving the barrier and same thing goes on.

Process	Code
P_0	1. $S = 0$ 2. $P_A = 1$ 3. $S = 1$ 4. $(1 \neq 3); \rightarrow$ condition true



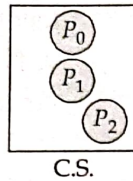
So, P_0 arrive at barrier and waiting for other process to arrive.

Process	Code
P_1	1. $S = 0$ 2. $P_A = 2$ 3. $S = 1$ 4. $(2 \neq 3); \text{ True}$



So, P_1 stop and waiting for other process.

Process	Code
P_2	1. $S = 0$ 2. $P_A = 3$ 3. $S = 1$ 4. $(3! = 3); \text{False}$



Now all 3-process arrived barrier, so while condition become false, to now the processes start leaving the barrier one by one.

Now suppose P_0 leaves the barrier, and not waiting for other processes (P_1 and P_2) to leave the barrier and immediately P_1 entering to barrier.

i.e.

Process	Code
P_0	5. $S = 0$ 6. $P_L = 1$ 7. $(1 == 3) : \text{false}$ (so, 8 and 9 do not execute) 11. $S = 1$

Now, $S = 1$

P_1 and P_2 still in loop.

So, we have two choices:

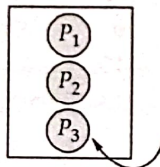
Choice 1:

P_1 can leave the barrier

Choice 2:

P_0 can enter into the barrier
(Case for deadlock)

Process	Code
P_1	1. $S = 0$
P_2	2. $P_A = 4$
P_3	3. $S = 1$
	4. $(4! = 3); \text{True}$



So, P_0 also enters the barrier and the while loop now, will be true forever because, we have 3 processes and all three processes stuck up at barrier.

Hence, no possibility of while loop to become false.

Hence, deadlock will occur.

Hence we conclude that, as soon as all the 3 processes arrive at barrier and until and unless, all the process leave the barrier, we will not call barrier again. So, two invocations if used in immediate succession, may lead to deadlock.

2.16 (b)

void barrier (void)

```

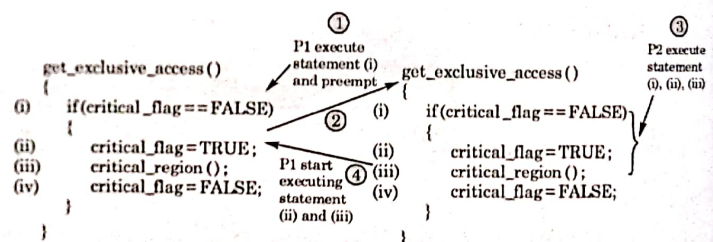
{
    while (process_left != 0);
    P(S);
    P_A++;
    V(S);
    while
    (P_A != 3);
    P(S);
    P_L++;
    if (P_L == 3)
    {
        P_A = 0;
        P_L = 0;
    }
    V(S);
}

```

Correction in implementation of code

2.17 (c)

Initially critical_flag = false



So, both process can enter in critical_region at same time.

So, no mutual exclusion.

We know that Dead lock $\rightarrow$ Mutual exclusion.

So, no Mutual exclusion $\rightarrow$ No deadlock. Here no deadlock possible.

Note: Here arrow shows order of execution and preemption is temporary, any process can preempt at any time.

2.18 (d)

In process P_1 if wants1 is true then wants2 entered in critical section and in process P_2 if wants2 is true then wants1 entered in critical section but in both cases a deadlock can occur but it ensures the mutual exclusion.

2.19 (c)

Writers code

wait (wrt)
writing is performed
signal (wrt)

• If read count == 1 then it must perform wait (wrt)
So, at same time reader and writer cannot access critical section.

• Since more than one reader can access the critical section simultaneously
So, at S2 signal (mutex) must be done.

Readers code

wait (mutex)
read count = read count + 1
if (read count == 1)
then wait (wrt) S1

signal (mutex) S2

reading is performed

wait (mutex) S3

read count = read count - 1

If (read count == 0)

then signal (wrt) S4

Signal (mutex)

Since by seeing S1 and S2 option (c) matches.
Hence it is correct answer.

2.20 (c)

In given code, P(S) decrements the value of semaphore and V(S) increment the value of semaphore. P_b and V_b are wait and signal operations on binary semaphore. X_b and Y_b are binary semaphore. To avoid the mutual exclusion condition if the value of X_b is 1 and the value of $Y_b = 0$ then P(S) and V(S) works properly.

2.21 (a)

- II. False let $P_1, P_2, \dots, P_n$ are processes let P_1 is currently holding lock and P_2 is waiting to enter CS after P_1 it is not definite that P_2 will surely enter critical section and it may happen that each time P_2 try to enter CS it is blocked by some other process which may lead P_2 to starvation.
 - III. False as there is not fixed order whichever process get lock access can enter so not necessary to follow FIFO.
 - IV. Given code of test and set instruction satisfy mutual exclusion so atmost one process can enter CS
- ∴ By elimination only option (a) is true.

2.22 (a)

P1	P2
While (S1 == S2);	While (S1 != S2);
Critical section	Critical section
S1 = S2;	S2 = Not(S1);

We initially take $S_1 = 1$, and $S_2 = 1$.

When P1 execute $S_1 = S_2$ then it do nothing and process P1 go into critical section after executing it make the value of $S_1 = S_2$ and when P1 execute then P2 not execute because that checks different condition in while so both show mutual exclusion. But progress is not shown by these two because in this one process can be delayed infinitely. Because again both P1 make its own condition true ($S_1 = S_2$ and $S_2 = \text{not}(S_1)$).

2.23 (a)

When $S_0 = 1, S_1 = 0$
So firstly the value of S_1 and $S_2 = 0$, P_1 and P_2 Not execute. The value of $S_0 = 1$, So it execute, process P_0 Now value of $S_0 = 0$
And it print zero one time when it release (S_1) call then value of S_1, S_2 increase by 1. So P_1 execute. It makes value of S_1 again zero and $S_0 = 1$, after it release (S_2) make value of $S_2 = 0$ by applying wait and $S_0 = 2$. So again P_0 process execute and wait (S_0) means it decrease by one so it print zero and this process continue. So P_0 print zero at least twice.

2.24 (b)

Assume P_1 executes until while condition and preempts before executing $L = 1$. Now P_2 executes all statements, hence $L = 0$. Then P_1 without checking L it makes $L = 1$ by executing the statement where it was preempted.
∴ It takes a non-zero value ($L = 1$) when the lock is actually available ($L = 0$)

2.25 (b)

Trick: To check deadlock in the given code proceed parallelly i.e. first executed the first line of all the processes then executed the second line of all processes and so on.

(a)	X	Y	Z
1.	P(a)	1. P(b)	1. P(c)
2.	P(b)	2. P(c)	2. P(d)
3.	P(c)	3. P(d)	3. P(a)

Value of a, b, c are 1 initially.
Execute first line of all processes i.e. $X_1 \rightarrow Y_1 \rightarrow Z_1$
after the executive of $X_1 \rightarrow Y_1 \rightarrow Z_1$ value of $\{a, b, c\} = 0$.
Now no process is allowed to enter into the critical section. So deadlock.

(b)

X	Y	Z
1. P(b)	1. P(b)	1. P(a)
2. P(a)	2. P(c)	2. P(c)
3. P(c)	3. P(d)	3. P(d)

1. Execute X_1 so value of b gets 0.
2. As value of b = 0 when we execute Y_1 it get blocked (Y is blocked).
3. Now execute Z_1 so value of a get 0.
4. Now try to execute the second line of unblocked processes as value of a = 0 when we try to execute X_2 it get blocked (X is blocked)
5. Now execute Z_2 , value of c gets 0.
6. As process X and Y are blocked so we can easily execute Z_3 .

So no deadlock, like wise we can check option c and d, so option (b) is correct.

2.26 (d)

	X	W	Y	Z
1.	P(s)	P(s)	P(s)	P(s)
2.	R(x)	R(x)	R(x)	R(x)
3.	$x = x + 1$	$x = x + 1$	$x = x + 2$	$x = x + 2$
4.	Store to MM	Store to MM	Store to MM	Store to MM
5.	V(s)	V(s)	V(s)	V(s)

1. Start with X. $P(s) \Rightarrow s = 1$ read $x = 0$, $x = x + 1$
Now before store into the memory process X preempt to process Y.
2. Y will perform $P(s) \Rightarrow s = 0$, read $x = 0$
 $x = x - 2$ then store $x = -2$ and V(s)
 $\Rightarrow s = 1$
3. Z will perform $P(s) \Rightarrow s = 0$, read $x = -2$
 $x = x - 2$
then store $x = -4$ and V(s) $\Rightarrow s = 1$
4. Now process x will store value of variable x into the MM so $x = 1$ and V(s) $\Rightarrow s = 2$
5. Now process W perform $P(s) \Rightarrow s = 1$ read $x = 1$
 $x = x + 1$, store $x = 2$, V(s) $\Rightarrow s = 2$,
Option (d) is correct

2.27 (c)

The solution is using a binary semaphore. X and Y are two different processes whatever the values inserted by the processes in the array that will be used by process Y afterwards.

Option (a): It is wrong, because there exists a deadlock.

Option (b): It is wrong, because process X can insert multiple values in an array and terminate but process Y can consume only one value.

Option (c): It is correct, because each and every value inserted by the process X in the array will be immediately consumed by process Y.

Option (d): It is wrong, because process X can insert two values in the array but the process Y can consume only one value if the preemption occurs after V(S) operation in the entry section of process Y.

Process Y:

```

-----
Entry Y(R, S) {
V(S);
-----

```

R = 0;
S = 1;

 → Preemption

Process X:

```

-----
Exists (R, S) {
V(R);
P(S);
-----
Exists (R, S) {
V(R);
-----

```

R = 1;
S = 0;

 → Preemption

Process Y:

```

-----
P(R);
-----
Entry Y(R, S) {
V(S);
-----

```

R = 0;
S = 1;

 → Preemption

Process X:

```

P(S);
-----

```

R = 0;
S = 0;

2.28 (c)

Option (a): If producer want to produce an item, it will first perform wait(s); then add to buffer(); then signal semaphore s, n

$\therefore$ Value of semaphore become

$$n = 1;$$

$$s = 1;$$

If we now run consumer code, we have to perform wait option on semaphore s, n

$$\therefore n = 0;$$

$$s = 1;$$

after removing from buffer(); it will signal(s) and consume()

$\therefore$ Consumer can consume item produced by producer so option (a) is false.

Option (b): It is also false as after removing one item from buffer we can repeat above process any number of times.

Option (c): If consumer run before producer has produced i.e. if buffer is empty, it will perform wait option on 's' which is successful after that it perform an unsuccessful wait option on semaphore 'n'

$$\text{So, } s = 1 \quad 0;$$

$$n = 0 \text{ (wait will lead to suspension)}$$

Now if producer try to run and perform wait(s); it will also get suspended as both process are suspended and no other process to signal them so deadlock occurs.

∴ Option (c) is true.

Option (d): False, if we consume item in the buffer using consumer code, followed by process in option (c), deadlock will occur.

2.29 (3)

B = 2

C = B - 1	D = 2 × B	C = B - 1	C = B - 1	D = 2 × B	D = 2 × B
B = 2 × C	B = D - 1	D = 2 × B	D = 2 × B	C = B - 1	C = B - 1
D = 2 × B	C = B - 1	B = 2 × C	B = D - 1	B = D - 1	B = 2 × C
B = D - 1	B = 2 × C	B = D - 1	B = 2 × C	B = 2 × C	B = D - 1
↓	↓	↓	↓	↓	↓
B = 3	B = 4	B = 3	B = 2	B = 2	B = 3

$$\Rightarrow B = 2, 3 \text{ or } 4$$

∴ Three distinct values are possible.

2.30 (a)

- (i) Mutual exclusion fail in the given program. First, process X executes all statements till varP = true. Second, process Y executes all statements till varQ = true; Now, both process X and process Y both can execute while condition as true and then enters critical section simultaneously. Therefore mutual exclusion fails.

- (ii) Deadlock is prevented by the given code.

2.31 (a)

Since pmax returns as integer not smaller than any of its argument there may be a case when $t[j] == t[i]$ and $t[j]$ is not equal 0 for it.

∴ while $(t[j] != 0 \ \&\& \ t[j] \leq t[i])$;

will be true and process may get stuck in infinite loop which lead to deadlock and hence progress is also not satisfied.

As there is not proper definite waiting for process.

∴ Among process $P_0 \dots P_{n-1}$ some process may not get chance to execute and may starve.

∴ Bounded wait is not satisfied.

2.32 (c)

Given problem is strict alteration problem it violates progress. As if initially when turn is initialized to zero and process 1 try to enter into critical section it will be stuck in while loop as a result condition for progress is violates because a process outside the critical section should not block another process from entering to critical section if it is free. Bounded wait is satisfied as after process 0 has executed process 1 will definitely get a chance to execute so there is bounded wait with respect to number of processes.

2.33 (7)

$$S - 20 + 12 = -1$$

$$S - 8 = -1$$

$$S = -1 + 8 = +7$$

So, the initial values of the semaphore should be '7'.

2.35 (c)

Full = N, empty = 0, mutex = 1

- Initially buffer will be empty, so consumer should not start first, so option b, D are eliminated.
- With option A consumer will never consume the item, so it is wrong.
- Option 'c' is correct answer which proper functionality of produce and consumer.

2.36 (80)

Initial value of D = 100

Minimum value occur when P1, P2, P3 read and P1, P3 update the value

$$P1 \text{ update } D = 110$$

$$P3 \text{ update } D = 120$$

and last P2 will update the $D = 100 - 50 = 50 = X$
Maximum value will occur when P2 and P3 read the initial value of D and update, first run P2 and update $D = 50$. P3 will run and update

$$D = 100 + 10 = 110$$

Now P1 will read the value of $D = 110$ and update

$$D = D + 20 = 110 + 20$$

$$D = 130 = Y$$

$$Y - X = 130 - 50 = 80$$

2.37 (a)

- Process 'p' is holding X_p resources and does not request any new resources. So 'p' can complete its execution and releases X_p resources.

- Process 'q' is holding X_q resources and does not request any new resources. So 'q' can complete its execution and release X_q resources.

Now available resources = $X_p + X_q$

- Other than 'p' and 'q' no other processes complete its execution, means available resources ($X_p + X_q$) is less than the resources demanded or requested by all other processes. So, $X_p + X_q < \min \{Y_k \mid 1 \leq k \leq n, k \neq p, k \neq q\}$ holds true.

2.38 (c)

All the processes running the given code will remain blocked due to wait(b) until the value of count becomes n .

When the value of count will become equals to n , value of b changes to 1 which will subsequently allow a process to go into section Q.

Hence, no process can go to section Q until all the process executes code section P.

2.39 (a, c, d)

- (a) True: (P_1) first process came executed wait(S); wait(S)

counter++; was partially executed and the process preempted. Other process came updated counter to 4.

Now P_1 came and wrote counter to 1. So (a) is correct.

- (b) Not possible at least counter will be 1.

- (c) True: All 5 processes executed first wait(S) and are blocked.

- (d) True: Each process executed the code sequentially and counter was finally updated to 5.

Option (a), (c) and (d) are correct.

2.40 (b, c, d)

- Mutual exclusion is not violated.
- Also, there will be no deadlock because of forceful preemption of resources.
- This may lead to starvation if the process is keeps on coming and preempting each other like P_1 is preempted by P_2 and P_2 is preempted by P_3 .
- Live-lock is also possible due to continuous preemption of resources.

For option (c) consider two processes P_1 and P_2 now P_1 enter the code acquires lock and resource. Now P_2 enters the else part kills P_1 and acquire R and restart P_1 .

Now P_1 again acquire lock and kills the process P_2 this continues creating a live lock scenario but there is ambiguity in the code since "Release R" is not written anywhere so ambiguity is regarding how the process will release Resource R. According to the code, the only way to release the resource is by getting killed.

2.41 (b, c)

P_1 and P_2 can spawn two threads T_1 and T_2 .

int $x = 0$; //global

Lock L_1 ; //global

main() {

foo(); //Thread T_1

foo(); //Thread T_2

print(x);

}

foo() {

a. int $y = 0$;

b. Acquire L_1 ;

c. $x = x + 1$;

d. $y = y + 1$;

Release L_1 ;

print(y);

}

- Let P_1 executed T_1 and in foo().

(a) $P_1 - T_1$ $\frac{x}{0} 1$

(b) $P_1 - T_1$

(c) $x = x + 1 (P_1 - T_1)$

.... Preempt T_1 of P_1 .

Similarly, perform thread T_2 of P_1 then $x = 2$.

Now, if we similarly perform both threads of P_2 then x will be maximum 2.

Note: It is not mentioned about IPC or concurrent mechanism so the process will not share the global variable. Each process will have its own address space so the maximum value of x by each process can be 2 only.

Note: But as we know every foo call will have its own copy of variable y so y can not be more than 1 in any case.

2.42 (c)

In order to get the required output only semaphore S_1 should be initialized to 1, other semaphore should be initialized to 0.

So, $S_1 = 1$; $S_2 = 0$; $S_3 = 0$

Answer is option (c).

2.43 (c)

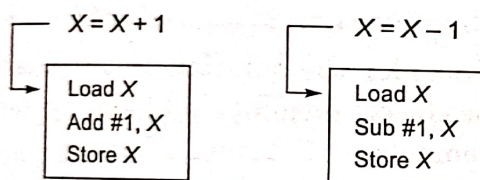
I-1: s is a binary semaphore initialized to 1. With binary semaphore, there can't be two threads that can simultaneously access the critical section. Thus, each thread with run $\text{incr}()$ or $\text{decr}()$ one by one.

Hence, only possible value of X at the end is 12.

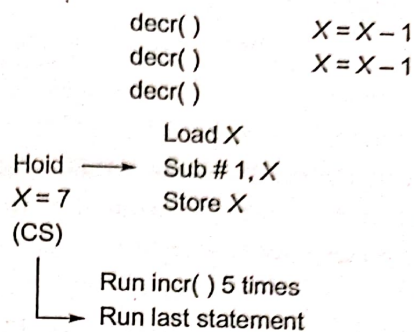
I-2: With counting semaphore of value 2.

Two processes at a time can access critical section.

Note:



In order to minimize X value, we need to maximize the impact of $\text{decr}()$ and minimize the impact of $\text{incr}()$.



Thus, finally $X = 7$ will get stored.

2.45 (a, b, c)

Option (d) is not possible to print *** continuously without printing \$ after **, all other option are possible.

■■■■